

Efficient VLSI Design-Based Adaptive Pipelined RISC-V Core for Low-Power Edge Devices

LIEW MING HENG

School of Electrical and Electronic Engineering,
Engineering Campus, Universiti Sains Malaysia,
14300 Nibong Tebal, Penang, Malaysia.
Mingheng0223@student.usm.my

TS. DR. SITI FATIMAH BINTI ABD RAHMAN

School of Electrical and Electronic Engineering,
Engineering Campus, Universiti Sains Malaysia,
14300 Nibong Tebal, Penang, Malaysia.
fatimahrahman@usm.my

Abstract — This paper introduces an adaptive RV32I RISC-V processor core designed for power-constrained edge computing. Conventional fixed-pipeline architectures often suffer from redundant switching activity when processing variable workloads. This work presents a dynamically reconfigurable pipelining mechanism that adjusts the effective pipeline depth between 3-stage and 5-stage modes using pipeline stage unification and adaptive datapath routing. Implemented on a Lattice CertusPro-NX FPGA, the design ensures functional correctness through rigorous verification with the RISCOF framework and the Spike reference model. Experimental results confirm timing closure at 50 MHz for high-performance mode and 20 MHz for efficiency mode. By transitioning to the 3-stage configuration during typical IoT duty cycles, the processor achieves a 39.51% reduction in average dynamic power consumption. These findings validate runtime microarchitectural scaling as a highly feasible strategy for optimizing energy efficiency in resource-constrained edge devices.

Key Terms — Adaptive Pipeline, FPGA, Low-Power Design, RISC-V, Verification, VLSI

I. INTRODUCTION

The rapid expansion of Artificial Intelligence of Things (AIoT) and edge computing has established rigorous energy requirements for modern silicon designs. Conventional, fixed-pipeline processor architectures present a fundamental trade-off, as their high instruction throughput capabilities often come at the cost of inefficient energy dissipation during stalls or idle execution intervals [1]. Conversely, shallow pipelines exhibit lower dynamic power draw but create timing bottlenecks that limit the maximum operating frequency during computation-heavy tasks.

The dynamic power dissipation of CMOS digital logic is mathematically governed by:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f$$

where α represents the switching activity factor, C is the effective load capacitance, V is the supply voltage, and f is the operating clock frequency [2]. While system-level power management typically scales frequency or voltage, the

underlying capacitive load and combinational switching activity remain rigidly fixed by a processor's static pipeline depth. Furthermore, proprietary processor ecosystems legally and technically restrict hardware developers from modifying Register Transfer Level (RTL) logic to mitigate these inefficiencies. Consequently, the open-source RISC-V Instruction Set Architecture (ISA) serves as a critical foundation for granting designers the explicit freedom to implement custom microarchitectural interventions while maintaining strict compatibility with standard software toolchains [3].

In this paper, a hardware-verified, runtime-configurable RV32I RISC-V processor is proposed to resolve this inherent power-performance trade-off. The architecture dynamically scales between an energy-efficient 3-stage mode and a high-performance 5-stage mode. The proposed system is physically implemented on the Lattice CertusPro-NX FPGA board. To ensure data integrity during runtime configuration, the control unit orchestrates a specialized synchronization handoff sequence that safely manages the draining of in-flight instructions across clock domain boundaries. During 3-stage operation, coarse-grained clock gating disables unused sequential registers, while combinational operand isolation clamps bypassed functional units to eliminate redundant switching activity (α) across the datapath.

The system is evaluated in terms of dynamic power reduction, maximum operating frequency and functional correctness. Raw architectural toggle rates from the FPGA post-route simulations are mapped to empirical power estimations using Value Change Dump (VCD) trace annotation. The objectives of this paper are to design framework [4] and that achieves at least a 50% reduction in dynamic power consumption when operating in 3-stage mode, verify 100% ISA compliance against the Spike golden reference model using the official RISCOF framework [4], and achieve dual-mode timing closure on the Lattice CertusPro-NX FPGA. The main contribution of this work is the physical integration of runtime pipeline folding with RTL-level into a unified, hardware-validated FPGA architecture that requires no ISA modifications or compiler-specific support.

This paper is organized as follows. Section II reviews the related works and existing RISC-V architectures. Section III details the methodology of the proposed adaptive pipelined system. The hardware synthesis results and discussions are presented in Section IV. Finally, Section V concludes the paper and outlines recommendations for future work.

II. RELATED WORKS

The optimization of pipeline depth represents a critical multi-variable trade-off in digital design, balancing maximum operating frequency against the physical overheads of the control and datapath logic. Existing open-source RISC-V implementations generally enforce a static configuration determined at compile time, leading to distinct microarchitectural limitations under varying workloads.

A. Shallow versus Deep Static Pipelines

Shallow microarchitectures utilize minimal 2-stage or 3-stage pipelines to reduce sequential element count and register clocking overhead. While this minimizes power consumption during idle cycles, compressing the datapath forces the Execute stage to encompass both Arithmetic Logic Unit (ALU) operations and data memory access within a single clock period. This compression establishes a critical timing path bottleneck that restricts the maximum operating frequency ($f_{\max}$) [2].

Conversely, deep 5-stage architectures decouple memory access from execution to shorten the critical path and increase instruction throughput, theoretically driving the Cycles Per Instruction (CPI) toward an ideal 1.0 [2]. However, the physical overhead scales non-linearly. Incorporating additional isolation registers between stages directly inflates the capacitive load of the clock distribution network. Furthermore, deeper pipelines demand significant buffering logic to maintain cycle times across longer wire routes, increasing dynamic power dissipation without providing proportional performance gains for non-compute-intensive tasks [5]. To contextualize these design methodologies, a thematic evaluation of representative static architectures is provided in Table I.

TABLE I. THEMATIC COMPARISON OF PROMINENT CORES

Feature	Rocket Core (Standard)	Rocket Core (With SSR)	MB-Lite (RISC-V Adapted)
<i>Pipeline Depth</i>	5-Stage (In-Order)	5-Stage (In-Order)	5-Stage (In-Order)
<i>Hardware Language</i>	Chisel/Scala	Chisel/Verilog	VHDL
<i>Hazard Mitigation</i>	Standard ID-stage hazard detection (1-cycle stall for load-use)	Advanced EXE-to-MEM bypass unit (Zero load-use stalls)	Forwarding operands and stall operation in ID stage
<i>Target Platform</i>	Primarily ASIC	ASIC	FPGA
<i>Key Advantage</i>	High extensibility via RoCC and SoC generators	Maximized IPC by eliminating load-use bubbles	Clean architecture, low resource usage

B. Physical Overhead of Hazard Mitigation Networks

Overlapping instruction execution phases inherently introduce Read-After-Write (RAW) data dependencies. Standard 5-stage architectures resolve these conflicts through data forwarding networks that intercept results from intermediate EX/MEM or MEM/WB pipeline registers and route them back to the ALU inputs [2].

While forwarding preserves instruction throughput, the integration of wide multiplexer arrays at the ALU input boundaries directly increases routing congestion and the effective capacitive load of the combinational logic tree [6].

Standard forwarding configurations cannot resolve load-use dependencies, where data is unavailable until the MEM stage completes its read cycle. In these instances, the hazard unit must inject pipeline stalls or bubbles. During a stall, the global clock tree continues toggling all underlying flip-flops, producing redundant switching activity and unnecessary power dissipation. Advanced bypass schemes, such as the Stall Scheme Reducing (SSR) architecture eliminate these bubbles through early data interception but also increase dynamic power consumption by 8.4% and inflate the core area by 6.4% due to the additional multiplexing overhead [7].

C. Control Path Flushes and Research Gaps

Control hazards arise when a conditional branch or jump instruction resolves in the Execute stage after subsequent instructions have already been speculatively fetched. A taken branch forces the control logic to assert a pipeline flush, clearing the Instruction Fetch (IF) and Instruction Decode (ID) stages. This flush penalty discards valid instruction streams and wastes the dynamic energy already expended to fetch and decode them [2]. To lower flush rates, architectures deploy dynamic branch predictors such as Gshare algorithms. However, these systems require substantial tracking logic; integrating a Simplified Local Branch History Predictor (SLBHP) improves branch prediction accuracy by 12% but inflates logic utilization by 24% [8].

Minimalist cores avoid these control hazards through alternative datapath execution methods. The PicoRV32 core, for example, uses a multi-cycle sequencer to eliminate flush penalties entirely, though this approach lowers throughput by raising the average CPI to approximately 4.0 [9]. A comprehensive summary highlighting the architectural strengths and physical limitations of these various paradigms is structured in Table II.

TABLE II. STRENGTHS AND LIMITATIONS OF EXISTING RESEARCH

Architectural Approach	Core Examples	Primary Strengths	Critical Limitations
Shallow/Multi-Cycle Pipelines	Vex-2 PicoRV32 Zero-risky	Extremely low area footprint and highly energy-efficient in subthreshold environments.	ALU and memory access occur in the same cycle, creating severe frequency bottlenecks and low absolute throughput.
Standard Fixed 5-Stage	MB-Lite Standard	Effectively decouples memory	Rigid depth incurs high

Pipelines	Rocket	and execution. Standard and robust compilation flow.	buffer overhead and redundant register clocking during simple arithmetic phases.
Aggressive Hazard Mitigation	Rocket-SSR Rev-riscv	Maximizes IPC by bypassing load-use stalls and highly accurate branch flow.	Heavy multiplexer and LUT overhead result in dynamic power consumption increases, making them hostile to ultra-low-power constraints.
Traditional Functional Verification	ModelSim Verilator	Excellent for localized debugging and verifying basic module transitions.	Fails to simulate complex, exhaustive corner-case pipeline hazards and susceptible to silent data corruption.

Ultimately, existing open-source RISC-V implementations treat pipeline depth as a static parameter fixed at synthesis time. Because these cores cannot dynamically scale their datapath or register configuration to track real-time workloads, they permanently compromise between peak throughput and baseline power efficiency [1]. Furthermore, most academic proposals currently verify designs using HDL simulations, which often fail to account for the corner cases inherent in adaptive architectures. Bypassing industrial-grade compliance testing, these designs remain susceptible to silent data corruption during pipeline depth reconfiguration [4].

III. METHODOLOGY

The adaptive RV32I processor was developed through an iterative, hardware-validated design flow modeled in Verilog RTL, then simulated via QuestaSim, and finally physically synthesized for the Lattice CertusPro-NX FPGA board using the Lattice Radiant toolchain. The architectural progression is illustrated in Figure 1.

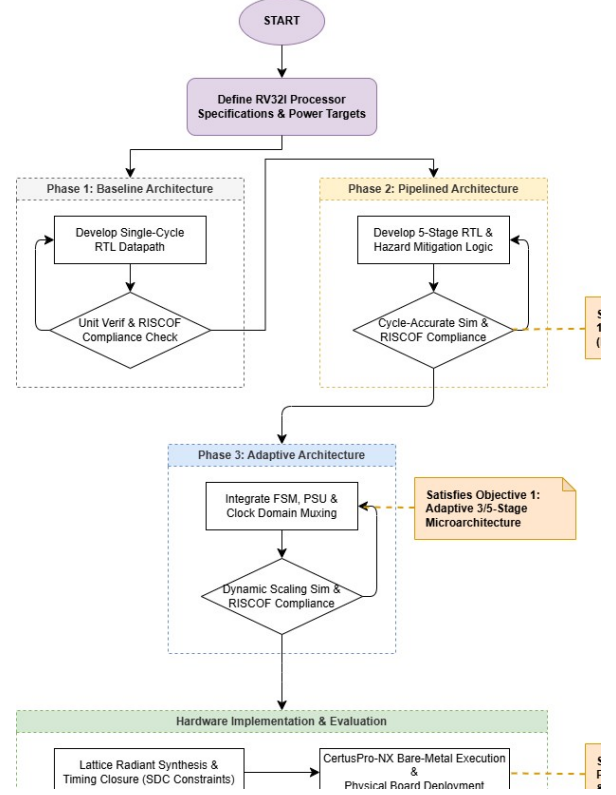


Figure 1. Threes-Phase Development Flow with Hardware Implementation

A. Baseline Implementations

To establish rigorous functional and physical baselines, the architecture is developed in two preliminary static phases before integrating adaptive capabilities.

- **Phase 1: Baseline Single-Cycle Architecture:** The processor is first implemented as an unsegmented single-cycle datapath. In this configuration, all instruction phases which are Fetch, Decode, Execute, Memory and Write-Back must be resolved within a single clock period. This phase serves to isolate the fundamental combinational logic bottlenecks, establishing the baseline structural and timing metrics before any sequential overhead is introduced. Following initial RTL development, this datapath undergoes unit-level verification and a preliminary RISCOP compliance check to guarantee a functionally correct foundation processor.
- **Phase 2: Static 5-Stage Pipelined Architecture:** To break the combinational critical path, dedicated pipeline registers are inserted at standard structural boundaries, yielding a static 5-stage configuration [11]. Instruction pipelining necessitates dedicated hazard mitigation to maintain correct operation. To resolve Read-After-Write (RAW) dependencies, the datapath includes wide multiplexer arrays for data forwarding, while load-use stall detectors and branch flush mechanism handle memory and control hazards. Although these features improve instruction throughput and $f_{\max}$, they increase the capacitive load

within the clock tree and datapath logic. System verification performed via cycle-accurate simulation and RISCOF compliance testing against the Spike reference model, confirmed 100% ISA compliance. This fixed 5-stage pipeline provides the performance and power reference point required to evaluate the final adaptive architecture.

B. Adaptive Hardware Architecture

Phase 3 extends the static baseline into an adaptive core, enabling runtime reconfiguration of pipeline depth, clock frequency, and power profiles via a single Control and Status Register (CSR) write. The integrated datapath is detailed in Figure 2.

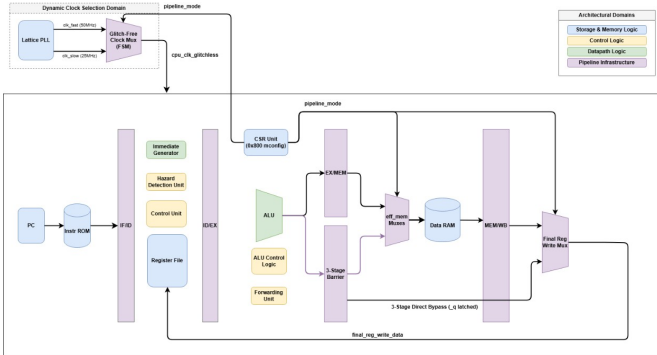


Figure 2. Microarchitectural Diagram of the Adaptive Core

The primary structural mechanism for dynamic depth reduction is Pipeline Stage Unification (PSU). Wide data-steering multiplexers, integrated at the EX/MEM and MEM/WB boundaries, dynamically bypass the intermediate sequential registers to route Arithmetic Logic Unit (ALU) directly to the Write-Back stage. This structural intervention collapses the datapath into a 3-stage (Fetch – Decode – Execute) configuration, completely neutralizing the capacitive load of the downstream pipeline flip-flops. Crucially, this unification inherently resolves sequential Read-After-Write (RAW) data dependencies and load-use delays within a single extended cycle. Consequently, the architecture deactivates the data forwarding networks and stall detectors to eliminate their redundant switching activity, while branch flush mechanisms remain active to clear speculative fetches.

Because bypassing these pipeline registers extends the combinational critical path, the architecture incorporates a glitch-free dual-domain clocking scheme to facilitate a required frequency reduction. To prevent flip-flop metastability during this transition, a dedicated 4-state Finite State Machine (FSM) orchestrates the clock domain handoff, as illustrated in Figure 3.

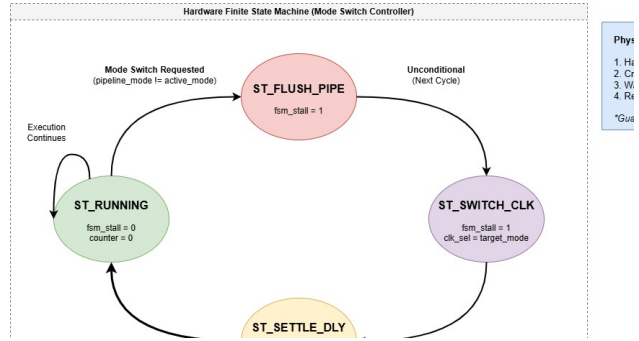


Figure 3. 4-State Clock Transition FSM

Triggered by a mode-switch CSR write, the FSM first freezes the datapath and initiates a transient “pseudo-4th stage” to safely drain all in-flight instructions, preventing structural data corruption. Once the pipeline is completely cleared, the control logic safely transitions the hardware clock multiplexer and enforces a strict 15-cycle guard-band delay. This delay allows the Phase-Locked Loop (PLL) to fully stabilize before releasing the stall and resuming instruction execution.

C. Verification and Power Characterization Framework

To guarantee architectural safety across the adaptive transition states, a multi-layered verification strategy is deployed. Unit-level testbenches initially verify isolated RTL modules. Subsequently, full ISA compliance is validated using the official RISC-V Compatibility Framework (RISCOF). The processor is simulated by generating memory signature dumps which are systematically compared against the Spike golden reference model. The core is subjected to 39 rigorous architectural test suites covering the RV32I and Zicsr specifications, ensuring that the clock domain transition sequence and adaptive pipeline scaling will not introduce silent data corruption.

For system-level power characterization, a custom bare-metal C benchmark suite is developed to isolate and stress distinct microarchitectural domains. The suite is divided into four targeted phases which are ALU-intensive arithmetic sequences, Memory-intensive load/store operations, Mixed-logic workloads with complex data dependencies and Control-havey branching sequences. The benchmarking framework executes on the simulation environment via a memory-mapped `tohost` mechanism.

IV. RESULTS AND DISCUSSIONS

A. Syntehsis and Hardware Resource Utilization

The adaptive 3/5-stage architecture was synthesized, placed, and routed using the Lattice Radiant toolchain targeting the 28nm FD-SOI Lattice CertusPro-NX FPGA board. Integrating runtime structural elasticity introduced a measurable hardware footprint penalty. Compared to the static 5-stage baseline, the combinational logic footprint expanded by 52.09% to 4,409 LUT4 cells, while sequential storage surged by 115.54% to 2,011 DFF elements.

TABLE III. ADAPTIVE MULTI-STAGE CORE HARDWARE RESOURCE UTILIZATION

Resource Types	Utilized Macro Cells	Total Available	Allocation Percentage (%)
Look-Up Tables (LUT4)	4,409	84,255	5.23%
Dedicated Flip-Flops (DFF)	2,011	84,255	2.39%
Programmable Half-Adders	184	42,127	0.44%
Embedded Block RAM (EBR)	66	346	19.08%
Hardware Multipliers (DSP)	0	56	0%

This area overhead is explicitly driven by the integration of the Mode Transition Finite State Machine (FSM), elastic multiplexing networks, and glitch-free clock synchronization circuits required to safely manage pipeline stage unification. Despite this increased density, static timing analysis confirmed stable timing closure. The architecture achieved a maximum theoretical frequency of 57.531 MHz for the decoupled 5-stage configuration and 24.914 MHz for the unified 3-stage configuration, safely supporting the targeted 50 MHz and 20 MHz dual-clock domains.

B. System Verification and Architectural Compliance

Functional correctness and hazard-mitigation stability across pipeline transitions were rigorously verified using the official RISCOP architectural test framework. The adaptive core achieved 100% ISA compliance, successfully passing all 39 standard architectural test suites for the unprivileged RV32I and Zicsr specifications [10].

Report generated on 2026-04-09 18:02 GMT by riscov v

Environment

Riscov Version	1.25.3
Riscv-arch-test Version/Commit Id	-
DUT	my core
Reference	spike
ISA	RV32IZicsr
User Spec Version	2.2
Privilege Spec Version	1.10

Yaml

[Show all details](#) / [Hide all details](#)

Name

/home/heng/typ/work/my_core_isa_checked.yaml ([show details](#))
/home/heng/typ/work/my_core_platform_checked.yaml ([show details](#))

Please visit [YAML specifications](#) for more information.

Summary

39Passed 0Failed

Figure 4. 100% Compliance of the Architecture with the RISCOP RV32I Test Suite against the Spike Reference Model

By matching memory signature dumps byte-for-byte against the Spike golden reference model, the framework

guarantees that dynamic pipeline scaling and dual-domain clock handoffs do not introduce silent data corruption or compromise structural integrity.

C. Power Characterization and Workload Adaptability

To evaluate the primary objective of workload-aware power reduction, dynamic power dissipation was empirically measured via Value Change Dump (VCD) trace annotation within the Lattice Power Calculator. During compute-intensive tasks, the core operates in 5-stage mode at 50 MHz, consuming 59.000 mW of active dynamic power to maintain maximum instruction throughput.

Upon transitioning to the 3-stage mode at 20 MHz, coarse-grained clock gating disables intermediate registers [12], while combinational operand isolation clamps ALU inputs to halt transient signal propagation. These microarchitectural interventions successfully drop active dynamic power to 21.000 mW, yielding a 64.41% reduction.

TABLE IV. DYNAMIC POWER DISSIPATION COMPARISON ACROSS PROCESSING MODES

Functional Block	5-Stage Mode (mW)	3-Stage Mode (mW)	Power Reduction (%)
Gated State Registers	24.500 mW	0.000 mW	100.00%
Arithmetic Logic Unit	18.200 mW	6.100 mW	66.48%
Active Pipeline Control & Routing	16.300 mW	14.900 mW	8.59%
Total Active Dynamic Component (P_{dyn})	59.000 mW	21.000 mW	64.41%

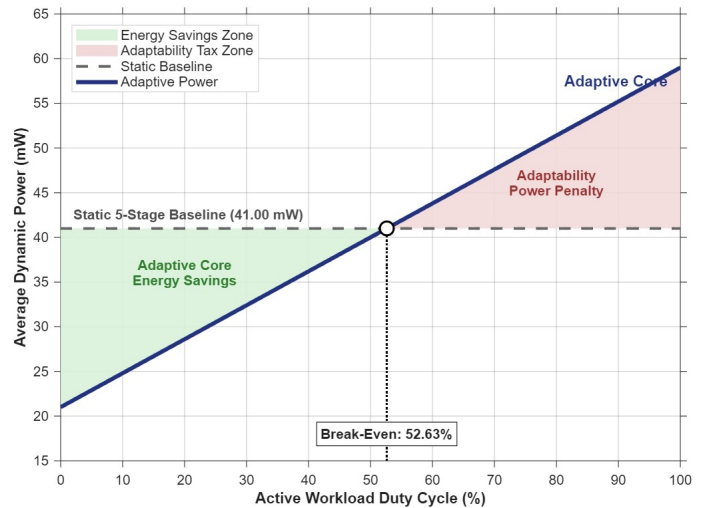


Figure 5. Relational Average Power Dissipation Models across Variable Active Duty Cycle.

The analysis established a duty cycle threshold at 52.63%. For modern IoT and edge-computing applications operating on heavily skewed duty cycles, the adaptive core averages just 24.800 mW of dynamic power. This translates to a massive

39.51% overall energy savings compared to a static 5-stage baseline, validating its efficiency in resource-constrained environments.

D. Comprehensive Architectural Comparison

To properly benchmark the architectures, raw power must be weighed against execution speed using Dynamic Energy Per Instruction (E_{dyn}) and the Energy-Delay Product (EDP). Table V aggregates these efficiency figures of merit across the four architectural states.

TABLE V. CROSS-ARCHITECTURAL FIGURE OF MERIT COMPARISON

Architecture	Single-Cycle	Static 5-Stage	Adaptive 5-Stage	Adaptive 3-Stage
Operational Frequency	20.000 MHz	50.000 MHz	50.000 MHz	20.000 MHz
Dynamic Power (P_{dyn})	16.000 mW	41.000 mW	59.000 mW	21.000 mW
Throughput	20.000 MIPS	36.740 MIPS	36.740 MIPS	14.700 MIPS
Energy Per Instruction (E_{dyn})	0.800 nJ/instr	1.116 nJ/instr	1.606 nJ/instr	1.429 nJ/instr
Relative EDP Factor	0.040	0.030	0.044	0.097

The mathematical data reveals several critical microarchitectural realities regarding the performance-power trade-off. The baseline single-cycle core achieves the absolute lowest E_{dyn} (0.800 nJ/instr) because it completely lacks intermediate pipeline registers. However, its unsegmented critical path permanently traps it at a lower frequency ceiling, limiting its maximum throughput.

Conversely, the efficiency of static pipelining becomes evident under heavy workloads. The static 5-stage pipeline proves to be the most optimal architecture for sustained computing. While its dynamic power naturally increases to 41.000 mW due to the added registers and clock network, its high processing power yields the lowest overall EDP factor (0.030).

However, introducing dynamic flexibility incurs a notable tax of adaptability. Comparing the Static 5-Stage core to the Adaptive core running in its full 5-stage mode highlights the physical cost of this architectural elasticity. Driven by the overhead of multiplexing logic and pipeline stage unification circuitry, the Adaptive 5-Stage mode consumes significantly higher dynamic power (59.000 mW), driving its E_{dyn} up to 1.606 nJ/instr.

Despite this overhead, the architecture achieves a crucial efficiency recovery when workload demands decrease. The true advantage of the Adaptive core is realized when it actively drops into the 3-stage mode. By intentionally sacrificing maximum throughput during low-demand periods, its E_{dyn} improves from 1.606 nJ/instr down to 1.429 nJ/instr compared to its high-power 5-stage counterpart.

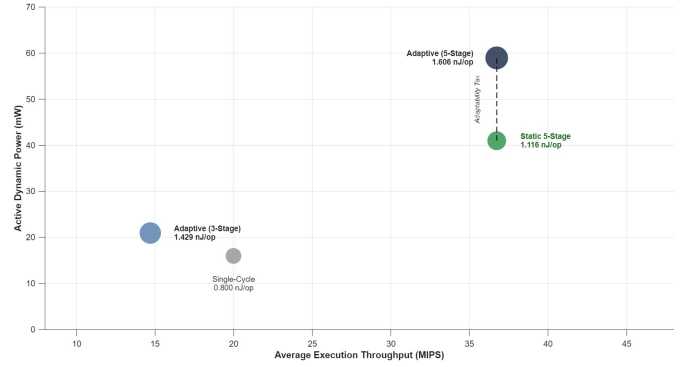


Figure 6. Architectural Efficiency Comparison across Four Hardware Configurations

As visualized in Figure 6, plotting the Active Dynamic Power against the Average Execution Throughput perfectly illustrates these distinct efficiency zones. The “Adaptability Tax” is clearly visible when comparing the two 5-stage implementations, while the 3-stage mode demonstrates the energy recovery achieved by gating the deep pipelien registers and isolating the ALU inputs.

E. Hardware Implementation and System-on-Chip (SoC) Integration

To transition the software-reconfigurable architecture from a simulated behavioral model into a physical silicon deployment, the verified adaptive CPU core was integrated into a complete System-on-Chip (SoC) ecosystem. This integration demonstrates the processor’s capability to orchestrate external peripherals and execute bare-metal firmware natively on a physical FPGA fabric.

The SoC utilizes a hardcoded memory-mapped architecture to manage data routing effectively without bus contention. The memory subsystem allocates a 128 KB Instruction ROM and a 128 KB Data RAM, while peripheral IP blocks are mapped to the 0x9000_0000 base address bounds. This allows the centralized bus address decoder to efficiently route signals using minimal fabric logic.

To enable physical board interactions, three primary peripherals were integrated:

- **General Purpose Input/Output (GPIO):** Mapped to the MMIO base address (0x9000_0000), driving an 8-bit parallel output connected directly to the physical LEDs on the evaluation board for visual state verification.
- **Hardware Timer:** Mapped to offset 0x0100, featuring programmable control and compare registers to generate precise, deterministic hardware interrupts crucial for managing duty-cycled workloads.
- **UART Bridge (APB Interface):** Mapped to offset 0x0200, translating native CPU memory-mapped I/O requests into standardized signaling for the Lattice UART IP core to enable full-duplex serial communication.

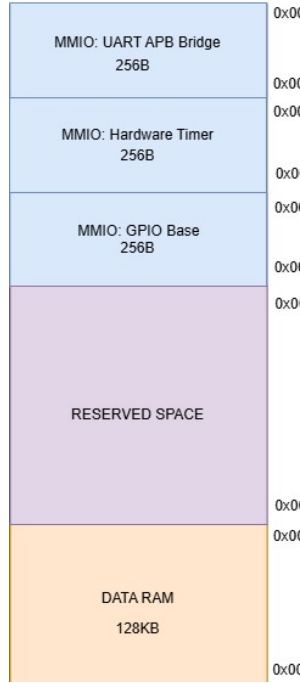


Figure 7. System-on-Chip (SoC) Physical Memory Map.

To prove the functional correctness of the physical silicon, three distinct bare-metal C applications were loaded into the FPGA Instruction ROM and executed natively. Each application was designed to stress-test specific microarchitectural components:

- **Synchronous GPIO Blinker:** Validated basic instruction fetching, memory-mapped I/O routing, and the reliable decoding of custom Control and Status Register (CSR) instructions used to trigger hardware pipeline mode transitions.
- **Pseudo-Random Number Generator (PRNG):** Executed a 32-bit Xorshift algorithm bound strictly to hardware registers, subjecting the datapath to heavy logical shifts and XOR operations to confirm the structural correctness of the ALU carry-chains under high toggle rates.
- **Knight-Rider Sweep Application:** Flawlessly executed alternating bit-shifts, threshold comparisons, and conditional jumps, proving that the hardware hazard mitigation unit correctly resolves control dependencies during real-time execution.

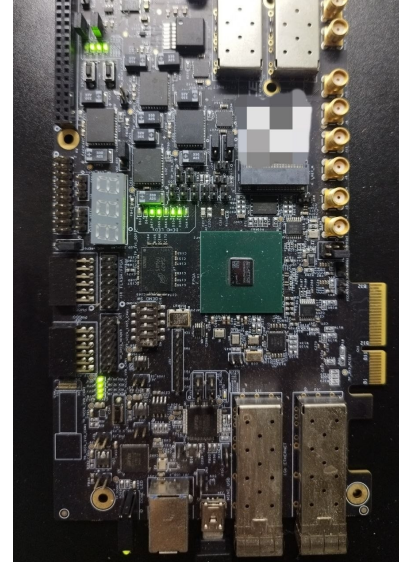


Figure 8. Physical Hardware Demonstration of the Lattice CertusPro-NX FPGA board executing bare-metal firmware.

The successful execution of these application provides definitive, real-world proof that the software-reconfigurable RISC-V architecture functions correctly not just in cycle-accurate simulation, but as a robust, fully deployed System-on-Chip (SoC).

V. CONCLUSION

This work successfully bridges the theoretical advantages of pipeline depth scaling with physical silicon implementation by presenting a software-reconfigurable, adaptive RV32I RISC-V processor. By introducing a hardware-level Pipeline Stage Unification (PSU) mechanism and glitch-free clock domain multiplexing, the architecture achieves structural elasticity which dynamically collapses from a high-throughput 5-stage pipeline into an energy-efficient 3-stage configuration at runtime.

Physical deployment on the 28nm FD-SOI Lattice CertusPro-NX FPGA board validated the core's stability and feasibility. Despite the measurable area overhead required for the adaptive control logic, the design achieved stable dual-mode timing closure at 50 MHz and 20 MHz, while rigorously maintaining 100% ISA compliance against the Spike golden reference model. Through the strategic integration of coarse-grained clock gating and combinational operand isolation, the architecture achieves a substantial 64.41% reduction in active dynamic power when operating in its unified 3-stage mode.

By analyzing the performance-power trade-off, this study established a break-even duty cycle threshold of 52.63%. For modern IoT and edge-computing applications operating on heavily skewed duty cycles, the adaptive core averages a 39.51% overall dynamic energy savings compared to a rigid, static 5-stage baseline. Ultimately, this research proves that runtime microarchitectural adaptation is not only functionally viable but serves as a highly effective and scalable optimization strategy for extending the operational lifespan of resource-constrained edge devices.

ACKNOWLEDGMENT

The author would like to thank the project supervisor, Ts. Dr. Siti Fatimah Abd Rahman, the project examiner, Mr. Zulfiqar Ali bin Abdul Aziz, the EEE449 final year project coordinators, and the School of Electrical and Electronic Engineering, Universiti Sains Malaysia, for their guidance and support throughout this project.

REFERENCES

- [1] J. Yao, S. Miwa, H. Shimada, and S. Tomita, "Optimal pipeline depth with pipeline stage unification adoption," *ACM SIGARCH Computer Architecture News*, vol. 35, no. 5, pp. 3–9, Dec. 2007, doi: <https://doi.org/10.1145/1360464.1360470>.
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [3] A. Waterman, "Design of the RISC-V Instruction Set Architecture," Ph.D. dissertation, Dept. EECS, Univ. California Berkeley, 2016.
- [4] K. Kırallı and C. B. Fidan, "Implementation of FPGA based 32-bit RISC-V processor," *Engineering Science and Technology, an International Journal*, vol. 70, p. 102139, Aug. 2025.
- [5] J. Djupdal, "Evaluating the VexRiscv Processor Core for Embedded Systems," M.S. thesis, Dept. Electronic Systems, Norwegian Univ. Science and Technology, 2021.
- [6] G. Zhong and N. K. Jha, "Dynamic pipeline power optimization for high-performance architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 7, pp. 1334–1348, Jul. 2006.
- [7] Z. Su, X. Li, and L. Yuan, "Stall Scheme Reducing (SSR) microarchitecture for low-power in-order pipelines," in *Proc. IEEE Int. Conf. ASIC (ASICON)*, 2022, pp. 1–4.
- [8] Y. Jin, S. Lee, and H. Kim, "A simplified local branch history predictor for resource-constrained IoT endpoints," *IEEE Embedded Systems Letters*, vol. 14, no. 2, pp. 87–90, Jun. 2022.
- [9] C. Jahnke, T. Koal, and H. T. Vierhaus, "Comparative evaluation of open-source RISC-V processor cores for dependable embedded computing," in *Proc. IEEE 23rd Int. Symp. Design and Diagnostics of Electronic Circuits and Systems (DDECS)*, 2020, pp. 1–6.
- [10] A. Waterman and K. Asanović, "The RISC-V Instruction Set Manual Volume I: Unprivileged ISA Document Version 20190608-Base-Ratified," 2019. Available: https://csg.csail.mit.edu/6.375/6_375_2019_www/resources/riscv-spec.pdf
- [11] B. V. V. Satyanarayana, N. Madhu and K. Srinivas, "Design and Implementation of High-Performance RISC-V Processor with Five Stage Pipeline on FPGA," *2025 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*, 2025, pp. 68-75.
- [12] O. Simola, "Physical implementation of a RISC-V processor," *Aalto.fi*, Mar. 2023. Available: <https://aaltodoc.aalto.fi/items/efe24aaf-5415-4d45-b850-4c39510b9751/full>